# Technical Reference

Overview of all modules and classes used in the GöROS framework along with their attributes and methods. For a broader explanation of each element's function, refer to the corresponding section in the documentation. For details on method parameters, refer to the method's docstrings.

This references uses the following aliases:

```
multiprocessing as mp
numpy as np
ev3_dc as ev3
```

## Code Structure

The repository containing the robot code has the following folder/file structure:

```
├──robot_code/
  ├── configs/
    ├── main_config.yaml
    ├── node_config.yaml
    └── topic_config.yaml
  ├── nodes/
    ├── __init__.py
    ├── camera_node.py
    ├── ev3_node.py
    ├── logger_node.py
    ├── main_node.py
    ├── node.py
    ├── robot_controller_node.py
    └── template_node.py
  ├── utils/
    ├── config_parser.py
    ├── config_utils.py
    ├── menu.py
    ├── message_utils.py
    ├── opencv_utils.py*
    └── print_utils.py
  ├── keypress_listener.py*
  ├── main.py
  ├── node_registry.py
  ├── publisher.py*
  ├── topic_registry.py
  ├── topic.py
  └── viewer_message.py*
├── subscriber.py*
├── drawables.py*
├── viewer_message.py*
└── viewer.py*
```

Files marked with * are retained from the legacy code and mostly needed for the operation of the viewer. They have not been documented in detail in this reference.

A table of contents for this reference can be found at the end of the document.

# Node

`file: nodes/node.py`

## Constructor

```
Node(
  name:str,
  main_config:Struct,       # the main config
  node_config:Struct,       # the node's config
  own_queue:mp.Queue=None,  # the node's command queue
  sys_queue:mp.Queue=None,  # the node's system queue
  send_queues:dict=None,    # command queues the node can access
  topic_configs:dict=None   # configs of the node's pub/sub topics
)
```

- the main and node config are assumed to be in the form of `Struct` objects, which can be addressed as `dicts`
- `send_queues` has the format `("node_name": Queue)`
- `topic_configs` has the following format:

```
{
  "pubs": {"topic_name": topic_config(Struct)},
  "subs": {"topic_name": topic_config(Struct)}
}
```

The init first calls `mp.Process.__init__()` (as `super()`) since this needs to happen before anything else[1], then it sets the class attributes, instantiates the topics, and configures printing.

## Attributes

### General Attributes

| Attribute | type | Function |
|---|---|---|
| `name` | `str` | Unique name of the node (`"camera"`, `"main"`, `"slam"`...). |
| `config` | `Struct` | Information on the node defined in `node_config.yaml`. |
| `mode` | `menu.Mode(Enum)` | Current mode, synced across the framework. |
| `last_mode` | `menu.Mode` | Previous mode, `None` on init. |
| `running` | `bool` | Node-specific, set `True` when `Node` process is started. |
| `count` | `int` | Counts iterations of the run loop. |
| `logging` | `bool` | Logging flag, set in `main_config.yaml`. |
| `print_lock` | `mp.Lock` | Allows printing without `BlockingIOException`. |

### Timekeeping Attributes

| Attribute | type | Function |
|---|---|---|
| `dt` | `float` | Desired length of one run loop iteration. Default: `DT=0.1 sec`. |
| `dt_warnings` | `bool` | Whether to warn when run loop exceeds `DT`. |
| `dt_buffer` | `float` | Fraction by which `DT` can be exceeded before causing a warning. |

[1] cf. documentation of the `multiprocessing` module here.

| Attribute | type | Function |
|---|---|---|
| `start_time` | `float` | Start time, set at start of `run()`, uses `time.monotonic()`. |
| `next_step` | `float` | Deadline for the current iteration of the run loop, uses `time.monotonic()`. |

### Queue-Related Attributes

| Attribute | type | Function |
|---|---|---|
| `sys_queue` | `mp.Queue` | The node's system queue. |
| `own_queue` | `mp.Queue` | The node's command queue. |
| `send_queues` | `dict{"n_name": Queue}` | Command queues the node can access. |
| `dummy_queues` | `list["n_name"]` | List of dummy queues. |
| `queue_threshold` | `int` | Max queue size before getting a warning. |
| `max_queue_msgs` | `int` | Number of msgs retrieved from command queue. |
| `is_dummy` | `bool` | Whether node was instantiated as dummy node. |
| `dummy_print` | `bool` | Whether to print any commands received when `self.is_dummy == True`. |

### Topic-Related Attributes

| Attribute | type | Function |
|---|---|---|
| `Node.pubs` | `dict{"t_name":Topic}` | Topics the node publishes to. |
| `Node.subs` | `dict{"t_name":Topic}` | Topics the node is subscribed to. |

## Methods

### Run Methods

```
Node.run()
```

Main run loop of the node. Gets triggered when the node registry calls a node's `start()` method (inherited from `mp.Process`).

```
Node.run_step()
```

Executes once per run loop iteration. Empty by default. The main place to add node-specific functionality.

### Queue Methods

```
Node.check_queue(threshold:int, max_msgs:int, sys_q:bool=False, flush:bool=False)
```

Retrieve messages from the node's system or command queue and run the received commands.

By default, all commands from the system queue are executed in each run step, but only a single command from the command queue. If the queue being checked has exceeded size `threshold`, the method prints a warning. Note that due to multiprocessing semantics, `mp.Queue.size()` is not reliable, so the threshold warning is not fully reliable either.

```
Node.send_to_queues(recipients:str/list, msg:tuple, log_msg:bool=True)
```

Send message (format `("method_name", tuple of args or None)`) to recipients' queues. If logging is enabled, every sent message is logged.

Note that if sending a tuple or list as the sole argument, this needs to be wrapped in a second tuple so it's not interpreted as a series of individual arguments!

```
# method inside "calculator" node:
def sum_tuple(self, numbers:tuple):
  return map(sum, list(numbers))

# calling this method from another node...
self.send_to_queues(
  recipients: "calculator",
  msg = ("sum_tuple", [1,2,3])
)
# ... will result in an error, since you are trying to call:
sum_tuple(1, 2, 3)

# Correct:
self.send_to_queues(
  recipients: "calculator",
  msg = ("sum_tuple", ([1,2,3],)) # comma after list to create a tuple
)
```

```
Node.flush_queue(sys_q:bool=False)
```

Flush the node's command or system queue.

Since `mp.Queue.size()` and `.empty()` are unreliable, flushing is achieved by retrieving every entry from the queue without executing any of them.

### Topic Methods

```
Node.topic_init(topic_configs:dict)
```

Initialize all topics the node publishes or subscribes to. Uses the `topic_configs` dict passed during `__init__()`, and returns a tuple of two dicts:

```
{"pubs":{"t_name":Topic`}, "subs": {"t_name": Topic}}
```

```
Node.publish_to_topic(
  topic_name:str,          msg: list/tuple/np.ndarray,
  pad_arrays:bool=False,   casting:bool=True)
```

Create a timestamp using `time.time_ns()` and publish message to the topic by calling `Topic.publish()`.

```
Node.read_topic(topic_name:str,max_msgs:int=0, new_only:bool=True) -> list/None
```

Read messages from the topic by calling `Topic.get_msg()`.

```
Node.get_synced_message(
  topic_name:str,      timestamp:int,
  threshold:int=0,     field_name:str="timestamp",
  direction:Literal["newer", "older", "nearest"]="nearest") -> tuple/None
```

Retrieve the topic's message that's closest to the reference timestamp by calling `Topic.get_synced_msg`.

### Logging Methods

```
Node.log(is_topic:bool, timestamp:int, rec, content=None)
```

Send content to the logger to be added to the log.

If logging is enabled, this method gets called every time a command is sent through queues or if a message is published to a topic. In the latter case, the logger is simply informed that publishing happened and consequently retrieves the message from the topic.

(**NOTE:** The logger is not yet fully implemented! Setting `logging=True` in the main config will result in the logger being included in the framework and nodes sending logging commands, but as of yet, these have no effect.)

### Shut-Down Methods

```
Node.stop_node()
```

Initiate graceful node shut-down by setting `self.running` to `False` and flushing any remaining commands from the system and command queues.

```
Node.shut_down()
```

Executes shut-down functionality by calling `Node.close_topics` (see below).

```
Node.close_topics()
```

Close all topics the node publishes or subscribes to, releasing their shared memory.

### Auxiliary Methods

```
Node.parse_command(msg:tuple)
```

Parse and execute a command message received through the node's queues. Command messages have the format `("method_name", tuple(args)/None)`.

If the function specified by the command does not exist, the method will print an error, unless the node is marked as a dummy node, then it will print the full command message to the console.

```
Node.ping()
```

Print a ping message.

```
Node.switch_mode(new_mode:int, flush:bool=True)
```

Save current mode to self.last_mode, (optionally) flush the command queue, and switch to new mode. The integers for the `Mode` enum are defined in `menu.py`.

## NodeRegistry

`file: node_registry.py`

### Constructor

```
NodeRegistry(t_reg:TopicRegistry, main_config:Struct, nodes_config:Struct)
```

This class uses the context manager pattern. To allow for an orderly shut-down of multiprocessing resources in case of an error, the nodes are not created inside `__init__()` but inside `__enter__()`.

The node registry is constructed after the topic registry, since it needs a reference to that to create the topic configs for each node.

## Attributes

| Attribute | type | Function |
|---|---|---|
| `t_reg` | `TopicRegistry` | Reference to the topic registry. |
| `main_config` | `Struct` | Contents of `main_config.yaml` parsed as `Struct`. |
| `nodes_config` | `Struct` | Contents of `nodes_config.yaml` parsed as `Struct`. |
| `nodes` | `dict{"n_name": Node}` | Dictionary of all nodes created by the registry. |
| `main_queue` | `mp.Queue` | Command queue for the `main` node. |
| `log_queue` | `mp.Queue` | Command queue for the `logger` node. |
| `system_queues` | `dict{"n_name": mp.Queue}` | Dictionary of all system queues. |
| `own_queues` | `dict{"n_name": mp.Queue}` | Dictionary of all command queues. |

## Methods

### Start-up and Shut-down Methods

```
NodeRegistry.__enter__() -> NodeRegistry
```

Enter the context and instantiate all Queues and Nodes. Following the context manager pattern, the method returns a reference to itself.

This method uses `nodes.get_all_nodes()` to find all node classes in the `nodes` module. If a node included in the framework does not have an associated class, it is instantiated as a dummy node.

```
NodeRegistry.run_nodes()
```

Start all nodes by calling `Node.start()` (inherited from `mp.Process`), which triggers `Node.run()`.

```
NodeRegistry.__exit__(exc_type, exc, tb)
```

Orderly exit the context by making sure all `Node` processes are finished and their resources are released. The arguments are required for the context manager pattern and are set automatically upon exit from the context.

### Setup Methods

```
NodeRegistry.create_queues()
```

Create system and command queues for each node and save the `main` and `log` queues separately to give the registry access to them.

```
NodeRegistry.set_send_queues(node_name:str) -> dict
```

Create a dictionary `{node_name: Queue}` of send-queues for the specified node.

If a node listed in the node config's list of queues was not included in the framework, the name of the node is added to the list of dummy queues in the node's config instead.

```
NodeRegistry.create_topic_config(node_name:str) -> dict
```

Create a nested dictionary that contains the topic-configs of each topic the given node publishes to or is subscribed to parsed as `Structs`. Dictionary format:

```
{
  "pubs": {"topic_name": topic_config(Struct)},
  "subs": {"topic_name": topic_config(Struct)}
}
```

### Auxiliary Methods

```
NodeRegistry.send_keypress(key:str)
```

Pass along a keypress character to the main node.

## TopType

`file: topic.py`

Enum for topic types. Currently has the values:

```
REG = 0   # registry copy, kept by TopicRegistry
PUB = 1   # publisher, only copy allowed to publish to topic
SUB = 2   # subscriber copy, can read topic
```

## Topic

`file: topic.py`

### Constructor

```
Topic(type:TopType, name:str, config:Struct)
```

The constructor sets the class attributes from the config and creates the message ring buffer. For this, the registry copy of the topic (`TopType.REG`) creates a block of `mp.SharedMemory`, and all copies set it as the buffer of an `np.ndarray`. The message head for the shared memory is administered by the publishing copy (`TopType.PUB`) and each topic keeps a local head to track which messages have been read.

### Attributes

| Attribute | type | Function |
|---|---|---|
| `name` | `str` | The topic's unique name. |
| `type` | `TopType` | The topic's type. |
| `config` | `Struct` | The topic's config. |
| `lock` | `mp.Lock` | Lock to prevent read/write conflicts. |
| `bl` | `int` | Length of the ring buffer |
| `publisher` | `str` | Name of the publisher `Node`. |
| `subscribers` | `list[str]` | Names of nodes subscribed to the topic. |
| `dtypes` | `Struct{"names[str],formats[str]"}` | Definition of message structure. |
| `shm` | `mp.SharedMemory` | The shared memory at the core of the ring buffer. |
| `buffer` | `np.ndarray` | The topic's ring buffer. |
| `head` | `int` | Local ring buffer index for publishing/ reading. |

## Methods

The methods of the `Topic` class are always called through either the `TopicRegistry` (on start-up), the `NodeRegistry` (when creating configs for the nodes), or through wrapper methods in the base `Node` class. They basically never need to be called directly inside node) functions.

## Shut-down Method

```
Topic.close()
```

Close the topic by releasing shared memory. If the topic is of `TopType.REG`, also unlink the shared memory.

## Publishing & Reading Methods

```
Topic.publish(timestamp:int, message, pad_arrays:bool=False, casting:bool=True) ->
bool
```

Publish a message to the topic's ring buffer after formatting and validating it against the topic's config. Returns `True` if publishing was successful, `False` if there was an error.

```
Topic.get_msg(max_msgs:int=0, new_only:bool=True) -> list/None
```

Retrieve messages (either a max number of unread messages or a set number of the newest messages) from the topic's ring buffer. The messages are returned as a list and the subscriber's local head is updated to the current message head.

```
get_synced_msg(
  timestamp:int,
  max_distance:int,
  field_name:str="timestamp",
  direction:Literal["newer", "older", "nearest"]="nearest",
  ) -> tuple:
```

Retrieve the message from the topic that's closest to the reference timestamp. If there is no message that fits the parameters passed as arguments (time difference does not exceed `max_distance` in the given direction (older/newer/nearest)), the method returns `None`.

You can specify a different field than the standard message timestamp that the target timestamp is compared against (e.g. if the data being published builds on data measured at an earlier point). In that case, you need to ensure that the alternative column uses the same function for creating timestamps (`time.time_ns()` (default timestamp), `time.time()`, or any other clock).

### Auxiliary Methods

```
Topic.search_timestamp(timestamp:int/float, msg_head:int, field_idx:int=0) -> int
```

Binary search to find the index at which the timestamp would be inserted into the ring buffer. While this method primarily operates on the standard `int` timestamp created by `time.time_ns()`, it can be used on any message field with a timestamp matching the format of the reference timestamp.

```
Topic.get_config() -> Struct
```

Return the topic's config.

```
Topic.get_subscribers() -> list
```

Return a list containing the names of nodes subscribed to the topic.

# TopicRegistry

`file: topic_registry.py`

## Constructor

```
TopicRegistry(topic_config:Struct)
```

This class uses the context manager pattern. To allow for an orderly shut-down of multiprocessing resources in case of an error, the topics are not created inside `__init__()` but inside `__enter__()`.

## Attributes

| Attribute | type | Function |
|---|---|---|
| `topic_config` | `Struct` | Contents of `main_config.yaml` parsed as `Struct`. |
| `topics` | `dict{"t_name": Topic}` | Dictionary of all topics. |

## Methods

### Start-up and Shut-down Methods

```
TopicRegistry.__enter__() -> TopicRegistry
```

Enter the context and instantiate the registry copy of every topic, i.e. the copy that creates the bock of `mp.SharedMemory` that will be used by every copy of the topic.

```
TopicRegistry.__exit(exc_type, exc, tb):
```

Orderly exit the context by closing all topics, meaning their shared memory is closed and unlinked. The arguments are required for the context manager pattern and are set automatically upon exit from the context.

### Pub & Sub Methods

```
TopicRegistry.read_message_from_topic(t_name:str, max_msgs:int=0,
new_only:bool=True) -> list/None
```

Read messages using the registry copy of a specific topic by calling `Topic.get_msg()`.

```
TopicRegistry.synced_message_from_topic(
  t_name:str,          timestamp:int,
  max_distance:int,    field_name:str="timestamp",
  direction:Literal["newer", "older", "nearest"]="nearest",
) -> tuple/None
```

Retrieve the topic's message that's closest to the reference timestamp by calling `Topic.get_synced_msg()`.

### Auxiliary Methods

```
TopicRegistry.has_topic(t_name:str) -> bool
```

Check whether a topic exists.

```
TopicRegistry.get_topic_config(t_name:str) -> Struct
```

Return the config of one specific topic (parsed as `Struct`).

# Mode

`file: utils/menu.py`

Enum for topic types. By default has the values:

```
IDLE = 0   # The framework initializes in Mode.IDLE
MANUAL = 1
```

# Menu

`file: utils.menu.py`

## Constructor

```
utils.Menu(main_config:Struct)
```

The constructor reads in the stop-key from the `main_config`, sets its own mode to `Mode.IDLE` (default at start-up), then sets an attributes containing a key press dictionary for each mode.

## Attributes

| Attribute | type | Function |
|---|---|---|
| `mode` | `Mode` | Mode of the framework, `Mode.IDLE` on start-up. |
| `last_mode` | `Mode` | Previous mode of the framework, `None` on start-up. |
| `reserved` | `dict`, see below | Dictionary of reserved key functions |

In addition to `Menu.reserved`, the `Menu` class has one key dictionary saved as a class attributes for every existing mode. These dictionaries contain the information about which key stroke will trigger which method depending on the mode the framework is in at the time. The format for the dictionary is:

```
{"key": ("recipient", "method_name", args)}
```

- `key (str)`: the key that triggers the command.
- `recipient (str/list[str])`: name(s) of target node(s) or “all”.
- `method_name (str)`: name of the target method.
- `args (single element/tuple/None)`: the argument(s) for the function call or `None` if no arguments are required.

An example for a key press dictionary might be:

```
self.manual = {
    'w': ("robot_controller", "move_man", ('w',)), # comma after 'w' to create
tuple
    'a': ("robot_controller", "move_man", ('a',)),
    's': ("robot_controller", "move_man", ('s',)),
    'd': ("robot_controller", "move_man", 'd'), # single elements are automatically
wrapped in tuples
    'f': ("robot_controller", "stop", None),
}
```

If a function takes a tuple or list as its only argument, that tuple needs to be wrapped in another tuple (cf. `Node.send_to_queues()`).

It's the sole responsibility of the programmer to make sure that no key is used multiple times inside one key dictionary. While `Menu.__init__()` checks that no reserved keys were used in a mode dictionary, there is no function to check for duplicates inside a dictionary.

### Methods

```
Menu.check_reserved_keys()
```

Ensure that no reserved keys are used in any of the mode dictionaries. This method is called automatically at the end of `Menu.__init__()` after all mode dictionaries have been set.

If you want to run this method to check your menu keys without booting the entire framework, you can use the test-method inside `main.py`:

```
def main_test():
    from utils.menu import Menu
    m = Main()
    menu = Menu(m.mc)

if __name__ == '__main__':
    main_test() # raises an error if a reserved key was used inside a mode dict
```

```
Menu.parse_keypress(key:str) -> tuple
```

Return a command and recipient node based on the current mode and the pressed key. This method returns the key press tuple defined in the mode dictionaries during `Menu.__init__()`.

```
Menu.swtich_mode(new_mode:int)
```

Switch the menu to a new mode and preserve information about the previous mode. This method gets called by the `main` node. Cf. `Mode` enum for each mode's integer values.

## Struct

`file: utils/config_utils.py`

This code was inspired by and partially adapted from the original code of the "Introduction to Computer Vision and Robotics" course.

### Constructor

```
Struct(data:Any=None)
```

The class `Struct` extends the Python-native `dict` object. When a `Struct` is created, the `dict` passed as an argument and all nested `dicts` are recursively turned into `Struct` objects.

### Attributes

All entries in the `data`-dictionary are set as values for the `Struct` object. If entries are added to, updated in, or deleted from the internal dict later, the corresponding class attribute will be added/updated/deleted and vice versa.

### Methods

```
Struct.wrap(value:Any) -> Any
```

Return value with every dict inside of it recursively turned into a `Struct` object.

Additionally, the `Struct` class extends some methods native to Python's `dict` object to keep the class attributes and entries in the internal dict in sync. These methods are:

```
Struct.__setattr__(name:str, value:Any) # set class attr. and dict entry
Struct.__setitem__(key:str, value:Any) # set dict entry and class attr.
Struct.__delattr__(name:str) # delete class attr and corresponding dict item
Struct.__delitem__(key:str) # delete dict item and corresponding class attr.
```

The following code illustrates some operations that are completely equivalent:

```
d = {"a": 1, "b": {"c": 2}}
s = Struct(d)

print(s.b.c)
print(s["b"]["c"])
print(s["b"].c)
# > 2

s.e = [3,4]
s["e"] = [3,4]

s.b.c = 6
s.["b"]["c"] = 6
s.["b"].c = 6
print(s.b) # > {"c": 6}

l = [(k,v) for k, v in s.items()]
print(l)
# > [("a", 1), ("b", ("c", 6)), ("e", [3,4])]
```

## Config Utils

`file: utils/config_utils.py`

This file contains methods that can be used to load the various configs. It calls the config parser, which is kept separately in `utils/config_parser.py`. When running this file as `__main__`, the `main()` function runs some checks on the topic config and calculates the memory size needed for all topics in the current implementation.

All configs are created as `Struct` objects, so their elements can be accessed like standard dictionaries entries (`main_config["configs"]`) or like class attributes (`main_config.configs`).

### Methods

```
load_config(filepath:str) -> Struct/None
```

Load a single config from a `yaml` file and return it as a `Struct`. Returns `None` if there was an error while loading.

```
load_all_configs((main_path:str, path_mod:str="") -> Struct
```

Load all configs from their respective `yaml` files, run them through the config parser, and return them collected in a single `Struct` object. The path for the main config is defined in `main.py`, the paths to all other configs are defined in `main_config.yaml`.

The `Struct` returned by the method uses as its keys the names defined for the different configs in `main_config.yaml`. If abbreviations are defined there, it uses those as keys.

```yaml
# main_config.yaml
configs:
  node_config:
    abbr: nc
    path: configs/node_config.yaml
  topic_config:
    abbr: null
    path: configs/topic_config.yaml
```

```
# load_all_configs() returns:
{
  "mc": Struct,           # containing main_config
  "nc": Struct,           # containing node_config
  "topic_config": Struct  # containing topic_config
}
```

The paths to other configs are given relative to `main.py`, since the framework is usually called from there. When calling the module from somewhere else, use the `path_mod` parameter to define the changed relative address. E.g. when running the `main()` method of this file by calling `python utils/config_utils.py` from the console, we set `path_mod = "../"` to address the folder structure:

```
robot_code/
├── configs/
    ├── main_config.yaml
    └── node_config.yaml
└── utils/
    └── config_utils.py
└── main.py
```

```
main()
```

This method is called when running `python utils/config_utils.py` from the console. It checks that all datatypes inside the topic config are valid and gives a read-out of each topic's size as well as the total size.

## Config Parser

`file: utils/config_parser.py`

This module contains functions that parse and update the various config files before the start of the framework.

The central method in this module is `parse_configs()`, which in turn calls all the individual methods needed for updating the configs. An overview of all updates and edits can be found at the very top of the `config_parser.py` file. If you want to add your own parsing methods, remember to add them to `parse_config()` as well!

All configs are assumed to be `Struct` objects, meaning their elements can be accessed like standard dictionaries entries (`main_config["camera"]["fps"]`) or like class attributes (`main_config.camera.fps`). They are edited in place, so all function arguments in this section are references to `Struct` objects.

## Methods

```
parse_configs(configs:Struct) -> Struct
```

Wrapper method that calls all parsing and update functions in order. If you want to add your own parsing methods, append them to the end of the function (e.g. in the Implementation-specific section).

### Parse Main

```
parse_main(mc)
```

Instantiate `print_lock` in main config.

### Parse Nodes

```
clean_nodes(nc:Struct) -> bool
```

Assert that each node listed under `include` in `node_config.yaml` has a node config. If that's not the case, the method returns `False` and the framework exits with an error. Configs for any nodes that are not included are deleted.

```
update_nodes(nc:Struct)
```

Add those parameters to all node configs that are set by the node registry (currently `is_dummy` and `dummy_queues`).

### Parse Topics

```
update_topic_configs(tc:Struct)
```

Update topic configs by

- adding `publisher` and `subscriber` elements to the topic config.
- replacing the generic buffer lenghts with specific values (set in `topic_config.yaml`).
- converting the format of multidimensional message fields from lists (needed by `yaml`) to tuples (needed by `numpy`).
- instantiating a `mp.Lock` for each topic config to avoid read/write conflicts.

```
check_pubs(nodes:Struct, tc:Struct) -> bool
```

Get the publisher for each topic from the node config and add them to the topic config. If a topic listed in a node config does not exist or a topic has more than one publisher, the method returns `Fales` and the framework exits with an error.

```
check_subs(nodes:Struct, tc:Struct) -> bool
```

Get the subscribers for each topic from the node config and add them to the topic config's list of subscribers. If a topic listed in a node config does not exist, the method returns `Fales` and the framework exits with an error.

```
find_orphan_topics(tc:Struct)
```

Give a warning if a topic has no publisher and remove all topics that have neither a publisher nor any subscribers.

### Implementation-specific Methods

These methods are used to update elements of the configs or replace placeholders when a config value is used in more than one place. E.g. both the `camera` node (captures images) and a potential `vision` node (detects markers in images) need access to the camera intrinsics, so any updates made to the intrinsics should happen during config parsing instead of each node having to perform the update individually.

```
update_camera(mc:Struct)
```

Reshape the camera matrix and distortion coefficients to match the dimensions given in the main config.

```
update_img(mc:Struct, tc:Struct)
```

Since several topics need to know the dimensions of the camera image (currently `camera_capture` and `marker_detections`), those values are defined in a central place in the main config. Instead, the topic config has placeholder variables `im_height` and `im_width`. This method replaces those placeholder variables with the values from the main config. If another topic requires the same replacement, it can be added to the `topics` dict at the start of the method

The method assumes that the dtype-format for the message field containing the image is given as `["im_height", "img_width", channels]`, e.g.

```
camera_capture:
    buffer_length: short
    dtypes:
      names: ["timestamp", "img", "cam_fps"]
      formats: ["int", ["uint8", ["im_height", "img_width", 3]], "int"]
```

## Message Utils

`file: message_utils.py`

This module contains a range of functions to check the formatting and validity of message configs and of the messages being published to a topic. It also has functions for formatting and padding multidimensional messages.

### Methods

#### Setup Methods

```
check_msg_fmt(t_name:str, dtypes:Struct) -> bool
```

Check whether a topic's message dtypes are formatted correctly, specifically:

- the number of field names and field formats is the same.
- each message field has a valid numpy datatype.
- the format for multidimensional fields is valid.

This method is called by the `config_parser` before the start of the framework. If it detects a problem, the framework exits with an error without starting the nodes.

```
calculate_memory_size(dtype_formats:list, bl:int) -> int
```

Calculate the memory requirements for a topic's ring buffer in bytes, based on its `dtypes` config.

This method is needed to calculate the fixed size of the `mp.SharedMemory` at the core of each topic. It is also called when calculating the total memory requirements of the current implementation by calling `utils/config_utils.py as __main__`.

#### Message Checking & Padding Methods

The methods in this section are called when messages are published to topics. They all assume that the `check_msg_fmt` was run when initializing the framework (this is currently built into

utils.config_parser.update_topic_configs(), so it's called automatically when the configs are loaded at the start of main.py).

```
format_message(
    t_name:str,
    timestamp:int,
    message,
    dtypes,
    pad_arrays:bool=False,
    casting:bool=True
) -> tuple
```

Assert that a given message conforms to dtypes in topic config and pad multi-dimensional message-fields where needed.

```
format_nd_field(
    t_name:str,
    f_name:str,
    f_fmt:tuple,
    value,
    pad_arrays:bool=False,
    casting:bool=True,
    msg:list=None
) -> np.ndarray
```

Check format of a multi-dimensional message field and add padding if needed. We currently use `empty` padding for arrays that are shorter than the defined field size, if you prefer a different padding type, you can change it near the end of the method.

Note: It is your responsibility to ensure that all elements of a multi-dimensional message field are of the same data type!

```
check_dtype(t_name:str, f_name:str, f_fmt:str, value, casting:bool=True, msg=None)
-> bool
```

Check whether the datatype of the value provided for a message field matches the data type set for that field in the topic config.

This method handles both standard Python types and `np.dtypes`.

#### Auxiliary Methods

```
print_error_message(t_name:str, error_msg:str, orig_msg=None)
```

Helper method used in this module to print errors that occur during message checking.

## Print Utils

`file: utils/print_utils.py`

This module contains a wrapper method that handles printing in a multiprocessing environment without running into BlockingIOExceptions. All files across the framework overwrite the standard `print()` method with `print_locked()`.

### Methods

```
configure_printing(lock:mp.Lock)
```

This method needs to be called once for every thread (i.e. once in the main process, once by each Node,

and once by the camera node's worker thread) and sets an `mp.Lock` that is shared by each of them. This lock ensures that only one process at a time tries to print to the console.

```
print_locked(
    *objects: Any,
    sep: str = " ",
    end: str = "\n",
    file: Optional[IO[str]] = None,
    flush: bool = False,
)
```

Print object(s) supplied via positional arguments without causing blocking errors. This function has an identical signature to `rich.print()` and the built-in `print()` and calls `rich.print()` in the background.[2]

## MainNode

`file: nodes/main_node.py`

The `main` node extends the basic Node class. It has to be included any time the framework is run, since it acts as the central administrative unit. The main node sends commands into each node's system queue, but it only has a single queue itself, which each node and the main process (through the node registry) can access.

### Constructor

```
MainNode(name:str, main_config:Struct, node_config:Struct, own_queue:mp.Queue,
sys_queues:dict)
```

The `main` node has a slightly simplified constructor compared to the base `Node`, since it only needs a single queue and no topic configs.

### Attributes

In addition to the standard Node attributes, the `main` node also has the following:

| Attribute | type | Function |
|---|---|---|
| `menu` | `Menu` | The `Menu` object that stores key press commands. |

The attribute `self.send_queues` contains the dictionary of system queues for all nodes.

### Methods

In addition to the standard Node methods, the `main` node adds/overrides the following:

```
MainNode.run()
```

Since the `main` node has no system queue, it only checks its (command) queue, retrieves all commands sent there and executes them. The `main` node has no `run_step()` method.

```
MainNode.check_queue(threshold:int, max_msgs:int, sys_q:bool=False,
flush:bool=False)
```

Handle calls to `check_queue()` with `sys_q=True` (happens e.g. when both queues are flushed during `Node.stop_node()`) without causing an error, since the `main` node's single queue is saved as `self.queue` and no system queue exists.

[2] The message signature and message docstring were adapted from `rich.print()`.

```
MainNode.parse_keypress(key:str)
```

Parse keyboard input according to the menu and forward commands to other nodes if necessary. The keystrokes are registered by the `Main` process and sent to the `main` node (via the node registry) as commands `("parse_keypress", "key")`. Key presses that have no function assigned to them by the `Menu` are silently discarded.

```
MainNode.switch_mode(new_mode:int)
```

Call the `Menu.switch_mode()` method and send the `switch_mode()` command to all nodes before calling it for the `main` node itself.

```
MainNode.stop_all()
```

Send stop command to all nodes, then execute own `stop_node()` method.

## CameraNode

`file: nodes/camera_node.py`

The `camera` node extends the basic `Node` class. The majority of the class is closely based on the original code written for the "Introduction to Computer Vision and Robotics" course. For now, we will only explain a limited number of its functions here.

### Constructor

```
CameraNode(
  name:str,
  main_config:dict,
  node_config:dict,
  own_queue,
  sys_queue,
  send_queues:dict,
  topic_configs:dict)
```

In addition to the standard constructor, when the `camera` node is created, it also loads various camera properties from the main config and creates a separate thread which handles the actual image capture.

### Attributes

In addition to the standard `Node` attributes, the `camera` node also has the following:

| Attribute | type | Function |
|---|---|---|
| `DEVICE` | `str` | Reference to the location of the camera. |
| `IM_WIDTH` | `int` | Width of camera images, set in main config. |
| `IM_HEIGHT` | `int` | Height of camera images, set in main config. |
| `FPS` | `int` | Maximum frame rate. |
| `exposure_time` | `int` | Exposure time, set in main config. |
| `gain` | `int` | Camera gain, set in main config. |
| `CAMERA_MATRIX` | `np.ndarray(float)` | Camera matrix, set in main config. |
| `DIST_COEFFS` | `np.ndarray(float)` | Distortion coefficients, set in main config. |
| `cap` | `cv2.VideoCapture` | Object that reads images from the camera. |
| `camera_thread` | `threading.Thread` | Thread that runs the camera capture. |

### Methods

In addition to the standard Node methods, the `camera` node adds/overrides the following:

```
CameraNode.run()
```

Extend the base `run()` function to add flags needed for the camera thread and initiate the same.

```
CameraNode.shut_down()
```

Extend the base `Node`'s `shut_down()` function to add orderly shut-down of the camera thread.

```
CameraNode.set_camera_properties(exposure_time:int=157, gain:int=50)
```

Connect to the USB camera and create `cv2.VideoCapture` object to read in images.

```
CameraNode.worker_thread()
```

Main camera thread. Publishes images and current frame rate to the topic `camera_capture` as frequently as fps allows. This is handled in a separate thread to the rest of the node, so it is not limited by the timing of the standard run loop.

```
CameraNode.warm_up()
```

Warm up the camera upon start-up.

```
CameraNode.close_camera()
```

Orderly shut down camera thread and release resources. Calling this method before shutting down the `camera` node is vital, since otherwise the connection to the camera might not be released and it will read as busy when the framework is started again. (If this does happen, waiting a while can help, and so can plugging the camera into a different USB port. If all else fails, restarting the Raspberry Pi usually solves the issue.)

———

The remaining methods in this class are legacy code from the original "Introduction to Computer Vision and Robotics" framework, which can be used to adjust the camera settings. They are not currently built into the framework, but are preserved in the file so you can include them or adapt them to your needs.

## EV3Node

`file: nodes/ev3_node.py`

The `ev3` node extends the basic Node class. It connects the framework to the LEGO EV3 controller and sends commands to the motors and receives input from the sensors connected to the same.

### Constructor

```
EV3Node(
  name:str,
  main_config:dict,
  node_config:dict,
  own_queue,
  sys_queue,
  send_queues:dict,
  topic_configs:dict)
```

In addition to the standard constructor, when the `ev3` node is created, it establishes a connection to the LEGO EV3 controller. If no controller is connected, the `ev3` node changes into dummy mode, meaning that commands that would normally need the EV3 controller to execute are either silently ignored or printed to the console.

### Attributes

In addition to the standard `Node` attributes, the `ev3` node also has the following:

| Attribute | type | Function |
|---|---|---|
| `wheel_radius` | `float` | Radius of the wheels in m, taken from main config. |
| `width` | `float` | Distance between centers of the wheels in m, taken from main config. |
| `ev3` | `ev3.EV3` | Object that allows communication with EV3 controller. |
| `vehicle` | `ev3.TwoWheelVehicle` | Vehicle object, used access motors. |
| `last_motor_pos` | `list[float, float]` | Last position of the two motors. |

### Methods

In addition to the standard `Node` methods, the `ev3` node adds/overrides the following:

```
EV3Node.run()
```

Publish an initial message to all topics the `ev3` node publishes to before calling standard `run()`. This is needed because normally, the various topics are only updated if there is a change, i.e. if the robot has moved, but e.g. the initial motor position is still needed for certain other methods.

```
EV3Node.run_step()
```

Once per run loop, publish messages to all topics if there has been a change.

```
EV3.move(speed:int, turn:int)
```

Send a movement command to the LEGO EV3 controller. The command makes the robot move at the given speed and turn for a short time. To achieve continuous driving, the method needs to be called repeatedly.

The speed is clipped at −100 (backwards) and 100 (forwards), turn is clipped at −100 (right) and 100 (left).

(This method is taken from the original code written for the “Introduction to Computer Vision and Robotics” course.)

```
EV3Node.stop()
```

Send a stop command to the EV3 controller to stop driving.

```
EV3Node.update_msgs(init:bool=False)
```

Update all EV3 topics (currently `wheel_position` and `motor_movements`) if there has been a change (unless `init=True`, then publish anyway). If no EV3 device is connected, this method will only publish an initial, empty message to each topic, then disregard any future calls.

```
EV3Node.stop_node()
```

Stop the EV3 vehicle if it is still running, then call the base `stop_node()` method.

```
EV3Node.switch_mode(new_mode:int, flush:bool=True)
```

Extend parent method so that when framework is in manual mode and no EV3 controller is connected, the normal dummy function of printing commands that can't be executed is turned off. (Otherwise there would be a constant stream of print-outs since the robot controller sends one `move()` command per $DT$.)

## RobotControllerNode

`file: nodes/robot_controller_node.py`

The `robot_controller` node extends the basic `Node` class. It handles the operational logic around the robot functions (currently: manual driving, later: gripper-operations, automatic drive controls, etc.) and sends the necessary commands to the ev3 node, which translates them into the format needed for the LEGO EV3 hardware. We explain our reasoning for keeping these functionalities separate rather than combining them into a single node in the documentation.

### Constructor

```
RobotControllerNode(
  name:str,
  main_config:dict,
  node_config:dict,
  own_queue,
  sys_queue,
  send_queues:dict,
  topic_configs:dict)
```

In addition to the standard constructor, the `robot_controller` node sets a number of attributes that are needed to administer driving functionality.

### Attributes

In addition to the standard `Node` attributes, the `robot_controller` node also has the following:

| Attribute | type | Function |
|---|---|---|
| `driving` | `bool` | Whether the robot is currently driving, init: `False`. |
| `speed` | `int` | Current driving speed in $[-100, 100]$, init: 0 |
| `turn` | `int` | Current turning speed in [−100 (right), 100 (left) ], init: 0 |
| `old_speed` | `int` | Previous driving speed, init: 0 |
| `old_turn` | `int` | Previous turning speed, init: 0 |

### Methods

In addition to the standard `Node` methods, the `robot_controller` node adds/overrides the following:

```
RobotControllerNode.run_step()
```

Control robot's behavior based on current mode. Currently only has functionality for `mode.MANUAL`.

```
RobotControllerNode.stop()
```

Send `stop()` command to ev3 node and update driving parameters.

```
RobotControllerNode.move_man()
```

Continuously send move() commands with current speed and turn to the ev3 node.

```
RobotControllerNode.set_move(char:str, speed_step:int=5, turn_step:int=5)
```

Set `self.speed` and `self.turn` based on keyboard input.

Pressing `'W'` while the robot is standing still (`speed=0`) or is moving forwards increases the movement speed by the `speed_step` up to a limit of 100. Pressing 'W' while the robot is moving backwards sets speed to 0.(`'S'` works the same for driving backwards.)

Pressing `'A'` while the robot is not turning (`turn=0`) or is turning left/counterclockwise increases the turning speed by `turn_step` up to a limit of 100. Pressing `'A'` while the robot is turning right/clockwise sets the turning speed to 0. (`'D'` works the same for turning right/clockwise.)

If `speed = 0` and `turn != 0`, the robot spins in place.

(This method is based on the original code written for the "Introduction to Computer Vision and Robotics" course.)

```
RobotControllerNode.switch_mode(new_mode:int, flush:bool=True)
```

Extend base switch_mode() method to add stopping when mode is changed.

## Main

`file: main.py`

This is the main process that you call to run the framework. It reads in the configs, instantiates the registries and the key listener, and sends data to the viewer.

At the top of `main.py` is the parameter `MAIN_CONFIG_PATH` that needs to point to the location of `main_config.yaml`. The locations of the other config files are defined within main config itself.

To start the framework, navigate to the `robot_code` folder in your console and run `python main.py`. This also provides an easy way to run tests without fully starting the framework (cf. below).

### Constructor

```
Main()
```

Loads the configs using the path specified at the top of the file and creates objects for the context manager classes (key listener, node and topic registries, and the `Publisher`) but does not start them yet.

The constructor also calls configure_printing() so safe printing is available across the entire main process (i.e. the `Main` class, the config utils, and all the context manager classes mentioned above).

### Attributes

| Attribute | type | Function |
|---|---|---|
| `n_reg` | `TopicRegistry` | The topic registry object. |
| `n_reg` | `NodeRegistry` | The node registry object. |
| `kl` | `KeypressListener` | The key listener object. |
| `publisher` | `Publisher` | Publisher object that sends messages to the viewer. |

| Attribute | type | Function |
|---|---|---|
| `dt` | `float` | Desired length of one run loop iteration. Default: `DT=0.1 sec`. |
| `dt_warnings` | `bool` | Whether to warn when run loop exceeds `DT`. |
| `dt_buffer` | `float` | Fraction by which `DT` can be exceeded before causing a warning. |
| `start_time` | `float` | Start time, set at start of `run()`, uses `time.monotonic()`. |
| `next_step` | `float` | Deadline for the current iteration of the run loop, uses `time.monotonic()`. |
| `last_marker_timestamp` | `int` | Timestamp of last message from `marker_detections` (needed for publishing to the viewer). |
| `running` | `bool` | Set `True` when `Main` process is started. |
| `count` | `int` | Counts iterations of the run loop. |

Additionally, any config loaded by the config utils is set as an attribute. For this, the config-abbreviation defined in `main_config.yaml` is used if one is given, otherwise the full name from the main config is used. At the moment, this means we currently have:

```
self.mc = main_config
self.nc = node_config
self.tc = topic_config
```

All entries in the `data`-dictionary are set as values for the `Struct` object. If entries are added to, updated in, or deleted from the internal dict later, the corresponding class attribute will be added/updated/deleted the same way and vice versa.

## Methods

```
Main.run()
```

Run loop of the main process.

The method enters the contexts for registries, publisher, and key press listener. Then, it sets `self.running` to `True` and enters the run loop. Once per `DT` (default: 0.1 sec), the run loop

- sends any key presses received to the `main` Node (via the node registry) to parse.
- calls the `pub_to_viewer()` method to publish data to the viewer.

```
Main.pub_to_viewer(count:int, timestamp:float)
```

Collect data from various topics and publish it as a message to the viewer.

This method accesses the topics through the topic registry and publishes null data if no new messages were found or the topics were not instantiated because they don't have a publisher.

The minimum requirement for the viewer to show any data is the presence of a camera image. All other values can be set to 0 if no data for them was found in the topics, but the camera image is required.

(This method is adapted from the original code written for the "Introduction to Computer Vision and Robotics" course.)

```
main_test()
```

This method can be used to test functions outside the run loop, e.g. config parsing, creation of the registries, etc. To use it, comment out `[Main().run()]` at the very bottom of `main.py` and comment in `main_test()` instead.

# Contents

Technical Reference ..... 1
Code Structure ..... 1
Node ..... 2
NodeRegistry ..... 5
TopType ..... 7
Topic ..... 7
TopicRegistry ..... 9
Mode ..... 10
Menu ..... 10
Struct ..... 11
Config Utils ..... 12
Config Parser ..... 13
Message Utils ..... 15
Print Utils ..... 16
MainNode ..... 17
CameraNode ..... 18
EV3Node ..... 19
RobotControllerNode ..... 21
Main ..... 22